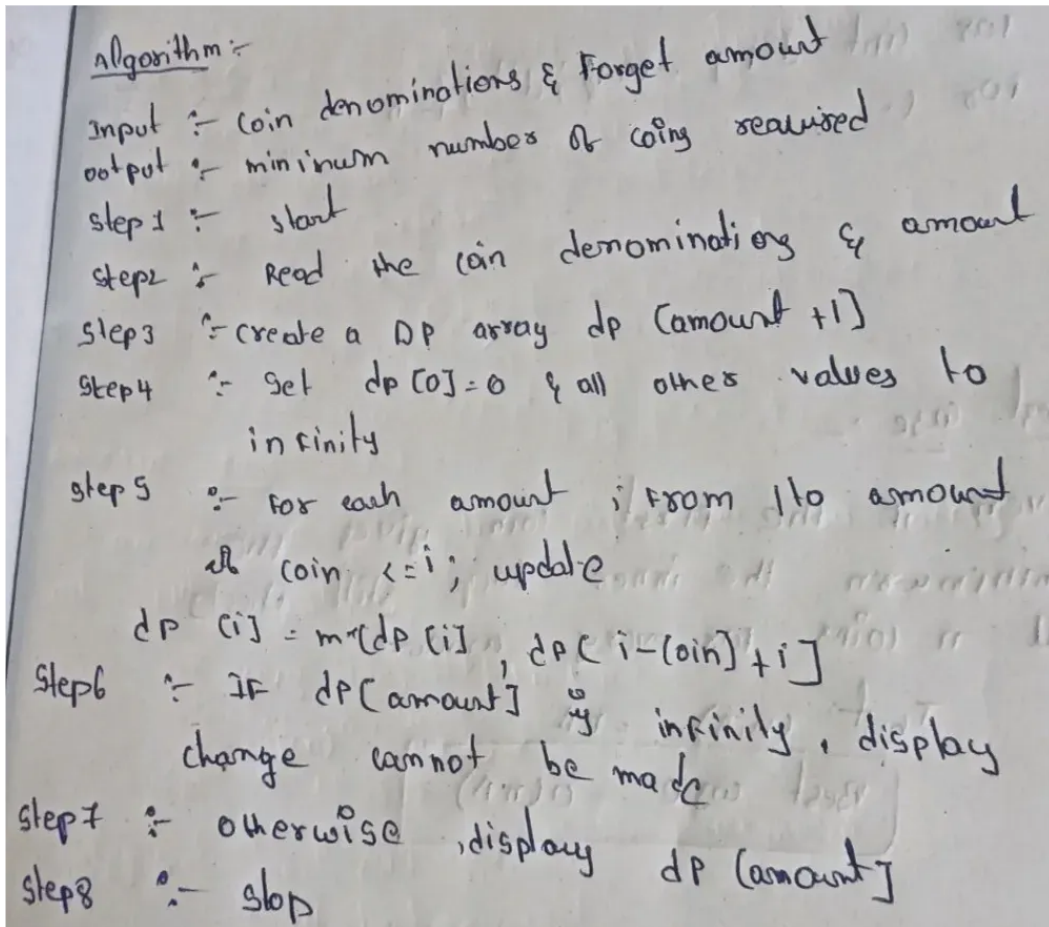


PRACTICAL-7

Aim : Implementation of making a change problem using dynamic programming

Algorithm :



Algorithm :-
Input :- coin denominations & Target amount
Output :- minimum number of coins required
Step 1 :- start
Step 2 :- Read the coin denominations & amount
Step 3 :- Create a DP array dp [amount + 1]
Step 4 :- Set dp[0] = 0 & all other values to infinity
Step 5 :- for each amount i from 1 to amount
 of coin <= i; update
 dp[i] = min(dp[i], dp[i - coin] + 1)
Step 6 :- If dp[amount] is infinity, display
 change cannot be made
Step 7 :- otherwise display dp[amount]
Step 8 :- stop

Program / Code :

```
#include <iostream>
#include <vector>
#include <climits>
#include <algorithm>
using namespace std;
```

```
int minCoins(vector<int> coins, int amount)
{
```

NAME: Raja Kiran

ENROLLMENT NO: 92400118654

```
// dp[i] = minimum number of coins needed to make amount i  
vector<int> dp(amount + 1, INT_MAX);
```

```
dp[0] = 0;
```

```
for (int i = 1; i <= amount; i++)  
{  
    for (int j : coins)  
    {  
        if (i >= j && dp[i - j] != INT_MAX)  
        {  
            dp[i] = min(dp[i], dp[i - j] + 1);  
        }  
    }  
}
```

```
if (dp[amount] == INT_MAX)  
    return -1;
```

```
return dp[amount];  
}
```

```
int main()  
{
```

```
    int n, amount;
```

```
    cout << "Enter number of coins: ";  
    cin >> n;
```

```
    vector<int> coins(n);
```

```
    cout << "Enter the coin denominations: ";  
    for (int i = 0; i < n; i++)  
    {  
        cin >> coins[i];  
    }
```

```
    cout << "Enter the amount: ";  
    cin >> amount;
```

```
    int result = minCoins(coins, amount);
```

```
    if (result == -1)  
        cout << "Change cannot be made for the given amount." << endl;  
    else
```

```
cout << "Minimum number of coins required = " << result << endl;
```

```
return 0;
```

```
}
```

Output :

```
Enter number of coins: 4
Enter the coin denominations: 2
5
3
1
Enter the amount: 500
Minimum number of coins required = 100
```

Time Complexity :

Time complexity

$$\text{for (int } d=1; d \leq n; d++) \{ \dots \} \rightarrow n$$

$$\text{for (int } i=1; i \leq n-d; i++) \{ \dots \} \rightarrow \frac{n(n-1)}{2}$$

$$i = i+d; \dots \rightarrow \frac{n(n-1)}{2}$$

$$\text{min} = 32767; \dots \rightarrow \frac{n(n-1)}{2}$$

$$\text{for (int } k=1; k \leq j-1; k++) \{ \dots \} \rightarrow \frac{n(n-1)}{2}$$

$$v = (T[j][k] + (C[k+1])(j) + P[i-1] * P[k] * P[j]);$$

$$\text{if } (v < \text{min}) \{ \dots \} \rightarrow \frac{n(n-1)(n+1)}{6}$$

$$\text{min} = v; \dots \rightarrow \frac{n(n-1)(n+1)}{6}$$

$$k = (i) \quad [j] = k \dots \rightarrow \frac{n(n-1)}{2}$$

$$\{ [i] [j] = \text{min} \} \rightarrow \frac{n(n-1)}{2}$$

$$\{ \dots \}$$

$$T(n) = n \cdot n \cdot \frac{(n+1)}{2} + n \cdot \frac{(n-1)}{2} + n \cdot \frac{(n-1)}{2} + 4 \cdot \frac{n(n-1)(n+1)}{6} + \frac{n(n-1)}{2}$$

$$T(n) = O(n^3)$$

1.BEST CASE:

Best case :-
Even if the first coin gives the minimum the inner loop still checks all n coins for every amount
 $T_{\text{best}}(n, A) = nA$

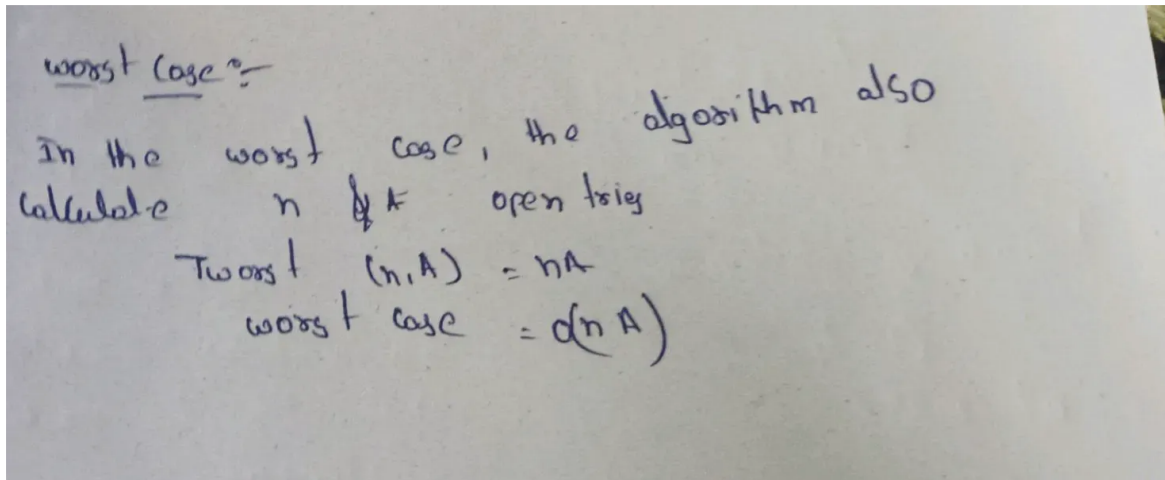
$\text{Best case} = O(nA)$

2.AVERAGE CASE:

Average case :-
on Average the algorithm still calculates all amounts & checks all n coin denomination
 $T_{\text{avg}}(n, A) = nA$

$\text{Avg case} = O(nA)$

3.WORST CASE:



Conclusion : The Coin Change problem was successfully implemented using Dynamic Programming to find the minimum number of coins. The $i \geq j$ condition ensures valid coin selection and avoids repeated calculations. The algorithm has $O(n \times \text{amount})$ time complexity and $O(\text{amount})$ space complexity, returning -1 when the amount cannot be formed.